Primer Parcial

Compresión de Imágenes mediante Quadtrees

Diseño y Análisis de Algoritmos

Alumnos:

- Borja Hernández Joshua
- Torres Hilario Andrik

Profesor:

Sánchez Sánchez Victor Manuel

Fecha: 09 / 10 /2026

REPORTE TÉCNICO | COMPRESIÓN DE IMÁGENES CON QUADTREES

A. Modelado del problema

Se aborda la representación compacta de imágenes RGB mediante subdivisión espacial. La entrada es una matriz de alto H, ancho W y tres canales de 8 bits (valores de 0 a 255), junto con un umbral de tolerancia $\tau \geq 0$. La salida es un árbol Quadtree serializable en un archivo binario .qtr, y una imagen reconstruida de las mismas dimensiones. El objetivo es disminuir el almacenamiento cuando existan regiones cromáticamente semejantes, permitiendo pérdida controlada de información.

La representación de cada nodo contiene coordenadas (x, y), ancho, alto, color RGB promedio si es hoja y una lista de hijos si se subdivide. Se admiten dimensiones impares y regiones rectangulares; cada subdivisión conserva únicamente cuadrantes no vacíos. Para los experimentos se emplearon imágenes sintéticas reproducibles: uniforme, cuatro regiones y ruido aleatorio (semilla 42). Las dimensiones evaluadas fueron 64×64, 128×128 y 256×256 píxeles.

B. Diseño algorítmico: Divide y Vencerás

El procedimiento analiza una región y calcula el color promedio por canal y el error cuadrático medio (MSE) entre los píxeles y dicho promedio. Si el MSE es menor o igual a τ, o la región tiene un solo píxel, se crea una hoja con el color promedio. En otro caso, se divide la región en hasta cuatro subregiones, se construye recursivamente un hijo para cada una y se conserva la estructura resultante. Este diseño sigue la descomposición espacial descrita por Finkel y Bentley (1974) y Samet (1984).

Etapa	Operación en el proyecto
Dividir	Separar el rectángulo en hasta cuatro cuadrantes válidos.
Resolver	Construir recursivamente un Quadtree por subregión.
Combinar	Conectar los hijos al nodo padre; no se fusionan píxeles.
Reconstruir	Rellenar cada región hoja con su RGB almacenado.

El formato .qtr contiene una cabecera de 16 bytes (firma QTR1, ancho, alto y número de bits), seguida de la estructura del árbol en preorden (0 = hoja, 1 = interno) y tres bytes RGB por hoja. La lectura recupera la geometría de los hijos a partir de las dimensiones, sin almacenar coordenadas individuales. Se comprueban firma, longitud y consumo de datos estructurales y RGB.

Criterio de uniformidad: $\text{MSE}(\text{región}) \leq \tau$. El umbral es local; el MSE final de la imagen completa se calcula después de reconstruir y puede diferir de τ. Para $\tau = 0$, las regiones no uniformes continúan dividiéndose hasta representar exactamente sus valores RGB.

C. Correctitud y análisis de complejidad

Justificación de correctitud

Invariante: el nodo correspondiente a cualquier región representa todos sus píxeles mediante una partición de hojas no superpuestas que cubre exactamente esa región. Caso base: si una región es hoja, la reconstrucción asigna su color almacenado a todos sus píxeles. Si $\tau = 0$, el MSE nulo implica que los valores de la región coinciden con el promedio; su reconstrucción es exacta. Paso inductivo: si una región se divide, los cuadrantes válidos forman una partición exhaustiva y disjunta. Suponiendo que cada hijo reconstruye correctamente su subregión, al colocar sus resultados en las posiciones originales se reconstruye la región padre. Por inducción estructural, se reconstruye toda la imagen; con $\tau = 0$ no existe pérdida de información.

La serialización escribe el tipo de cada nodo en preorden y el RGB de cada hoja. La deserialización lee la misma secuencia y deriva los cuadrantes con la misma función de división; por inducción sobre el árbol, recupera su estructura y colores. Las pruebas de guardado/carga y reconstrucción verificaron esta propiedad en los casos ensayados.

Recurrencia y Teorema Maestro

Sea $N = H \times W$ el número de píxeles. Para una imagen cuadrada, subdivisiones equilibradas y cuatro hijos en cada nodo interno, calcular promedio y MSE de una región de N píxeles cuesta $\Theta(N)$. En el peor caso todas las regiones se subdividen: $T(N) = 4T(N/4) + \Theta(N)$, con $T(1) = \Theta(1)$.

Teorema Maestro: $a = 4$, $b = 4$ y $N^{\log_4 4} = N$. Como $f(N) = \Theta(N)$, se cumple el caso 2 y $T(N) = \Theta(N \log N)$. Intuitivamente, cada nivel procesa en total $\Theta(N)$ píxeles y existen $\Theta(\log N)$ niveles en la descomposición equilibrada. En el mejor caso se reconoce la imagen como una sola región y el costo es $\Theta(N)$, por la evaluación inicial de todos los píxeles.

Operación / recurso	Costo asintótico	Condición
Construcción (mejor)	$\Theta(N)$	Una región homogénea.
Construcción (peor)	$\Theta(N \log N)$	Subdivisión completa equilibrada.
Reconstrucción	$\Theta(N)$	Cada píxel se escribe una vez.
Árbol almacenado	$O(N)$	Hasta $\Theta(N)$ nodos y hojas.
Pila recursiva	$O(\log N)$	Cuadrantes cuadrados equilibrados.

La reconstrucción crea una matriz RGB de N píxeles, por lo que requiere $\Theta(N)$ memoria de salida. El árbol utiliza $O(N)$ espacio en el peor caso, más arreglos temporales empleados por NumPy. Para regiones extremadamente estrechas o particiones degeneradas, las cotas de profundidad deben interpretarse según la geometría efectiva; la formulación del Teorema Maestro corresponde al caso cuadrado equilibrado.

Pruebas de correctitud y robustez

Con tolerancia cero se verificó igualdad exacta entre original y reconstrucción para los tres tipos de imágenes. Una prueba de dimensiones impares de 7×5 píxeles produjo 53 nodos y 35 hojas, con reconstrucción exacta. Además, el lector rechazó correctamente tres archivos alterados: cabecera inválida, datos RGB truncados y bytes RGB sobrantes. Estas pruebas no equivalen a una verificación exhaustiva de todos los archivos posibles.

D. Resultados experimentales

Se ejecutaron 27 configuraciones: tres tipos de imagen × tres resoluciones × tres tolerancias (0, 2 000 y 5 000). Se utilizó Python en Google Colab; los tiempos de construcción y reconstrucción se midieron mediante la mediana de cinco repeticiones. Los resultados completos, con 15 variables por registro, se entregan en resultados_experimentales.xlsx y .csv. Las duraciones dependen del equipo y de la carga del entorno.

Tabla 1. Construcción con tolerancia cero

Imagen	64 × 64 (ms)	128 × 128 (ms)	256 × 256 (ms)
Uniforme	0.6406	0.6781	2.4190
Cuatro regiones	0.4325	1.3353	5.0175
Aleatoria	171.4336	606.0797	2577.2816

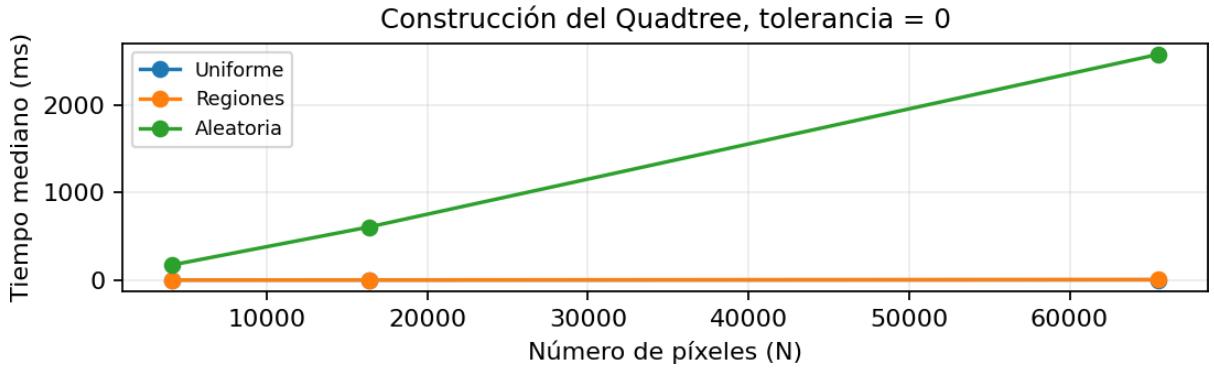


Figura 1. Tiempos medianos medidos con tolerancia cero; datos del archivo experimental generado en el proyecto.

Las imágenes uniforme y de cuatro regiones se resuelven con uno y cinco nodos, respectivamente, para las tres resoluciones. En contraste, la imagen aleatoria de 256×256 y τ = 0 generó 87 381 nodos y 65 536 hojas; su tiempo de construcción fue 2 577.2816 ms. El crecimiento observado es compatible con el análisis de costo, pero tres tamaños de prueba no constituyen una demostración empírica de una cota asintótica.

Tabla 2. Calidad y almacenamiento, imagen aleatoria de 64 × 64

τ	Archivo .qtr	Ahorro vs. RGB	MSE global	Hojas
0	12 987 B	−5.69 %	0	4 096
2 000	12 398 B	−0.90 %	94.1973	3 910
5 000	3 953 B	67.83 %	3 370.7383	1 243

El tamaño base para comparar es la matriz RGB sin comprimir (12 288 bytes), no un archivo PNG o JPEG. El ahorro se calculó como 100 × (1 − bytes_qtr / bytes_RGB); la razón de compresión como bytes_RGB / bytes_qtr. El MSE se evaluó por píxel y canal RGB después de reconstruir la imagen.

D. Interpretación y E. Conclusiones

Interpretación de los experimentos

En las imágenes uniformes, el Quadtree requiere una sola hoja y el archivo ocupa 20 bytes, incluso a 256×256 píxeles; esto evidencia su eficacia cuando hay grandes áreas homogéneas. Para las imágenes de cuatro regiones, el árbol se estabiliza en cinco nodos y el archivo ocupa 29 bytes. En ambos casos, la reconstrucción con $\tau = 0$ es exacta.

En imágenes aleatorias, la tolerancia cero exige almacenar casi todos los píxeles en hojas: para 64×64, el archivo .qtr resulta 5.69 % mayor que los bytes RGB originales. Con $\tau = 5\,000$, el almacenamiento cae a 3 953 bytes (67.83 % de ahorro), pero el MSE global aumenta a 3 370.7383. Por tanto, la compresión obtenida depende del contenido y del error visual aceptable; un mayor ahorro no implica necesariamente una mejor representación.

E. Conclusiones

Se implementó una solución reproducible basada en Divide y Vencerás que construye, serializa y reconstruye imágenes RGB mediante Quadtrees. La estrategia adapta el número de regiones a la variabilidad cromática, admite dimensiones impares y recupera sin pérdida las imágenes evaluadas cuando $\tau = 0$. El análisis formal establece $\Theta(N \log N)$ para la construcción en el peor caso cuadrado equilibrado, $\Theta(N)$ en el mejor caso y $\Theta(N)$ para la reconstrucción.

La técnica resulta especialmente conveniente para imágenes con extensas regiones de color constante o similar. Sus limitaciones incluyen la sobrecarga del árbol en imágenes con ruido, el costo de recalcular estadísticas regionales, la pérdida de calidad con umbrales elevados y la necesidad de un lector específico del formato .qtr. Como mejoras se proponen sumas acumuladas para acelerar estadísticas, tolerancias adaptativas, codificación adicional y comprobaciones de integridad más fuertes.

Uso de inteligencia artificial como herramienta de apoyo

Durante el desarrollo se utilizó ChatGPT (OpenAI) como recurso de asistencia académica y técnica. Su participación incluyó explicaciones de Divide y Vencerás y del Teorema Maestro; apoyo para estructurar el notebook; propuestas y revisión de fragmentos de código; orientación en la depuración, serialización, diseño de pruebas y presentación de tablas, gráficas y documentación. Las propuestas se trataron como material sujeto a revisión y se contrastaron mediante ejecuciones y resultados observables en Google Colab. La IA no se considera una fuente de mediciones experimentales ni sustituye las pruebas del proyecto.

Reproducibilidad y materiales entregados

El repositorio incluye quadtree.py, experimentos.py, el notebook proyecto_quadtree.ipynb, README.md, requirements.txt, resultados CSV/Excel y tres pares de evidencias PNG/.qtr (uniforme, cuatro regiones y aleatoria). Las instrucciones de ejecución se encuentran en el README. El repositorio será publicado en GitHub por el equipo (GitHub, s. f.); el entorno de experimentación fue Google Colab (Google, s. f.).

Alcance: los datos reportados corresponden a imágenes sintéticas con semilla fija y a las configuraciones ensayadas. No se afirma superioridad frente a PNG o JPEG, pues no se realizó una comparación experimental directa con dichos formatos.

Referencias bibliográficas

- Finkel, R. A., & Bentley, J. L. (1974). Quad trees: A data structure for retrieval on composite keys. Acta Informatica, 4(1), 1–9. https://doi.org/10.1007/BF00288933

- GitHub. (s. f.). About repositories. GitHub Docs. https://docs.github.com/en/repositories/creating-and-managing-r epositories/about-repositories

- Google. (s. f.). Google Colab: Frequently asked questions. https://research.google.com/colaboratory/faq.html

- Matplotlib Development Team. (s. f.). Quick start guide. Matplotlib Documentation. https://matplotlib.org/stable/users/explain/quick_start.html

- NumPy Developers. (s. f.). numpy.mean. NumPy Documentation. https://numpy.org/doc/stable/reference/generated/numpy.mean.html

- OpenAI. (2022, 30 de noviembre). Introducing ChatGPT. https://openai.com/index/chatgpt/

- Python Software Foundation. (s. f.). struct — Interpret bytes as packed binary data. Python Documentation. https://docs.python.org/3/library/struct.html

- Samet, H. (1984). The quadtree and related hierarchical data structures. ACM Computing Surveys, 16(2), 187–260. https://doi.org/10.1145/356924.356930